
Energy-Efficient Docking Station Location and Route Planning for Autonomous Underwater Vehicles

Project Report

EMU676 Optimization Models and Algorithms in
Transportation and Distribution

Nazlı Elif HANOĞLU

June 10, 2026

Hacettepe University

Contents

1	Introduction	3
2	Problem Description	4
3	Problem Formulation	6
3.1	Node-Based Formulation	7
4	Metaheuristic Algorithm	11
4.1	Solution Representation and Battery Model	12
4.1.1	Chromosome Representation	12
4.1.2	Battery Simulation and Full-Recharge Model	12
4.2	Initial Population	12
4.2.1	Greedy Construction	12
4.2.2	Diverse Initial Population	13
4.3	Greedy Dock Insertion	13
4.4	Genetic Algorithm Operators	14
4.4.1	Selection	14
4.4.2	Crossover: Order Crossover (OX)	14
4.4.3	Mutation	14
4.5	Local Search	15
4.5.1	2-opt	15
4.5.2	Relocate	15
4.5.3	Exhaustive Dock Search	15
4.6	Diversification	15
4.7	Overall Algorithm	16
5	Experimental Analysis	19
5.1	Experimental Setup	19
5.2	Benchmark Instances	19
5.3	Computational Results	20
5.4	Mathematical Model Performance	21

5.5	Docking Station Utilisation	21
5.6	Genetic Algorithm Performance	22
5.6.1	Scalability Analysis	22
5.7	Comparative Discussion	23
5.8	Solution Visualisation: Instance S3	24
6	Conclusion	26

Chapter 1

Introduction

Traditional methods for underwater inspection relied on human divers or manned submersibles, which are costly, limited in duration and depth, and may have negative environmental effects [1]. To improve such operations, Autonomous Underwater Vehicles (AUVs) can be employed for marine operations, including marine research, military reconnaissance, seabed mapping, pipeline and cable inspection, and environmental monitoring.

One of the main limitations in AUV operations is energy availability, since these vehicles typically rely on batteries with limited capacity. The amount of energy consumed directly affects mission duration, coverage area, and overall operational effectiveness. For this reason, finding routes that minimize energy usage is crucial while ensuring that all target points are visited efficiently [2].

The use of underwater docking stations supports long-term sustainability through battery recharging and data communication. These stations enable vehicles to recharge their batteries and transfer collected data without needing to return to the surface [3]. The location of docking stations plays a critical role in determining optimal routes. Therefore, it is important to consider the location of docking stations and energy-efficient routing strategies together to improve operational efficiency and reduce costs.

Chapter 2

Problem Description

This project focuses on the inspection of critical subsea pipeline infrastructure and seafloor mapping using Autonomous Underwater Vehicles (AUVs). The main objective is to determine the most cost-effective routes for one or more AUVs while simultaneously deciding where to locate underwater docking stations to overcome battery range limitations.

In the literature, several researchers have explored energy-efficient routing strategies for underwater vehicles. For instance, energy-efficient route planning methods have been applied in underwater pipeline inspection tasks using resident AUV systems [2]. In addition, heuristic optimization techniques—such as genetic algorithms, ant colony optimization, and particle swarm optimization—have been proposed to improve location and route planning in complex and dynamic underwater environments.

Kasparavičiūtė and Ludvigsen [1] introduced an location and routing optimization using a distance-constrained Vehicle Routing Problem (VRP) solved via Genetic Algorithms (GA). Their approach ensures that AUVs complete high-priority subsea inspection tasks while maintaining sufficient energy to return to a charging station. Another study, the Resident Autonomous Underwater Vehicle Routing Problem (RAUVRP) has been proposed to optimize pipeline inspections, emphasizing the use of Subsea Docking Plates (SDPs) as critical nodes for both energy replenishment and data transfer [2].

De Carolis et al. developed an energy-aware estimation framework that uses runtime metrics to update mission plans via an Energy-Aware Orienteering Problem [4]. This allows vehicles to adapt to external factors like sea currents, which directly impact battery consumption.

Furthermore, the stochastic nature of the underwater environment introduces significant uncertainty into route planning. To mitigate this, robust global route optimization have been developed using greedy-strategy-based Genetic Algorithms (GGA) [5]. These heuristic techniques are particularly effective in balancing the trade-off between task completion and risk management in dynamic settings.

This research is motivated by the growing need for long-term underwater monitoring

and inspection systems, where AUVs are expected to function autonomously for prolonged periods. By optimizing both the location of the docking station and energy-efficient routing strategies, it is possible to extend mission duration, reduce operational expenses, and improve the reliability of autonomous underwater systems.

This project aims to:

1. Formulate a mathematical model as a location-routing optimization problem with energy constraints for the AUVs.
2. Develop algorithms such as genetic algorithms or other metaheuristics to find feasible solutions.

Chapter 3

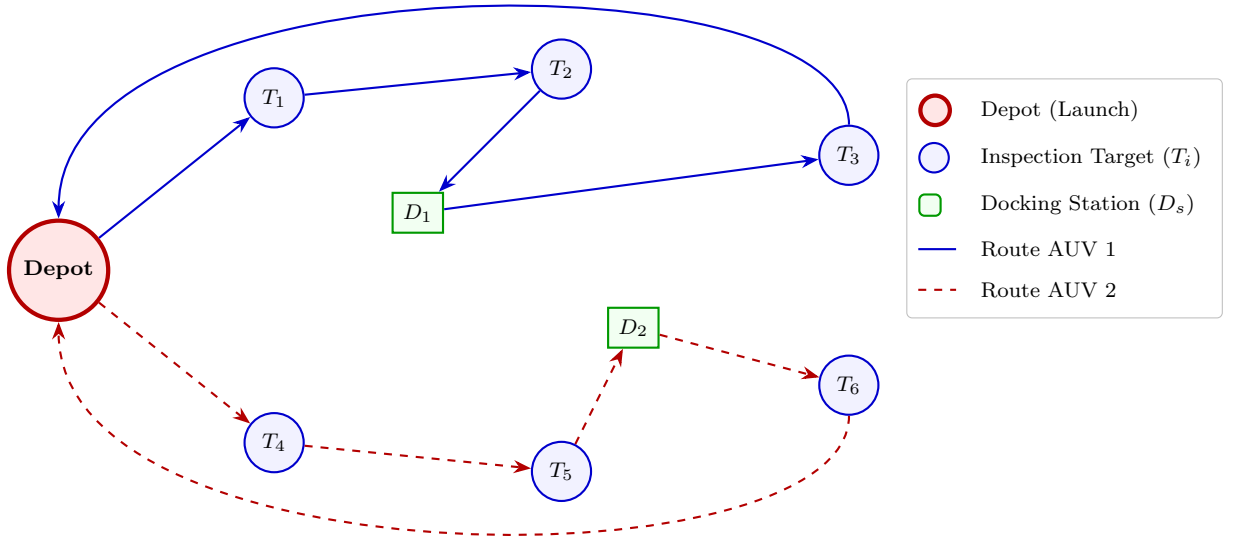
Problem Formulation

The model is formulated as a **Mixed-Integer Linear Programming (MILP)** problem that seeks to minimize the total distance traveled by all AUVs and fixed costs associated with installing and maintaining underwater charging stations. In order to ensure the AUV's inspection missions are effective and reliable, some operational constraints must be considered:

- **Full Inspection Coverage:** Every target inspection node (P) on the pipeline must be visited exactly once by a single AUV.
- **Energy and Battery Management:** AUVs operate under strict energy constraints due to water resistance and the power demands of inspection sensors. The battery level must never drop below a safe threshold and must always be sufficient to reach the next available charging point or the home depot.
- **Starting and Ending Points:** The depot is the starting and ending point for all AUV missions.

This problem can be modeled as both node-based and arc-based formulations each serving a specific functional role:

- **Node-Based Perspective:** The node-based model focuses on the state of the AUV at specific coordinates. It monitors the nodes (locations) themselves, rather than the movement between them. It is primarily responsible for tracking localized constraints, such as the battery level. In pipeline inspection, this guarantees that the AUV maintains enough energy levels at each critical vana or sensor site.
- **Arc-Based Perspective:** The arc-based model focuses on the movement and flow between locations. This is the primary mechanism for calculating total operational costs and ensuring "flow conservation". For subsea missions, this structure optimizes the global path across the pipeline network.



Graph-based representation of the AUV routing problem. Nodes represent inspection points, while square nodes indicate candidate docking stations. Two AUV routes are illustrated with different line styles. Each route starts and ends at the depot and may include docking visits for recharging. UAV that visits any dock copy is assumed to recharge to full capacity Q upon arrival. This eliminates the incentive to visit the same physical dock consecutively (since a single visit already provides a full charge) and simplifies the battery update constraints.

3.1 Node-Based Formulation

Graphical Notation

The problem is modeled on a directed graph $G(V, A)$ where V is defined as nodes, and A is as arcs: $A = \{(i, j) : i, j \in V, i \neq j\}$.

Sets and Indices

- $V = \{0, 1, \dots, n\}$: All nodes; node 0 is the depot
- $P \subset V$: Pipeline inspection nodes
- $D \subset V$: Candidate docking station nodes (physical locations)
- $D' = \{j^1, j^2, \dots, j^{m_j} \mid j \in D\}$: Expanded set of dock *copies*; each physical dock j is replicated m_j times to allow multiple visits. All copies of dock j share the same location, fixed cost f_j , and open/close decision y_j .
- $V' = \{0\} \cup P \cup D'$: Expanded node set used in the routing model

- K : Set of UAVs
- $A = V' \times V'$: Set of all directed arcs over the expanded node set

Parameters

- d_{ij} : Euclidean distance between node i and node j (for dock copies j^c : $d_{i,j^c} = d_{i,j}$)
- c : Travel cost per unit distance
- Q : Maximum battery capacity of the UAV
- q_i : Energy cost of performing inspection at node i ($q_i = 0$ for dock copies and depot)
- f_j : Fixed cost of opening physical docking station $j \in D$
- M : A sufficiently large constant (Big-M)

Decision Variables

$$x_{ijk} = \begin{cases} 1 & \text{if UAV } k \text{ travels from node } i \text{ to } j \in V' \\ 0 & \text{otherwise} \end{cases}$$

$$y_j = \begin{cases} 1 & \text{if docking station } j \in D \text{ is opened} \\ 0 & \text{otherwise} \end{cases}$$

$$z_{jk} = \begin{cases} 1 & \text{if UAV } k \text{ charges at dock } j \in D \\ 0 & \text{otherwise} \end{cases}$$

$$e_{ik} \geq 0 \quad : \text{ battery level of UAV } k \text{ upon } \textit{departure} \text{ from node } i \in V'$$

$$u_{ik} \geq 0 \quad : \text{ MTZ auxiliary variable for node } i \in V'$$

Objective Function

$$\min \sum_{k \in K} \sum_{(i,j) \in A} c d_{ij} x_{ijk} + \sum_{j \in D} f_j y_j \tag{3.1}$$

Subject to

$$\sum_{k \in K} \sum_{j \in V'} x_{ijk} = 1 \quad \forall i \in P \quad (1)$$

$$\sum_{j \in V'} x_{ijk} = \sum_{j \in V'} x_{jik} \quad \forall i \in V', \forall k \in K \quad (2)$$

$$\sum_{j \in V'} x_{0jk} = 1 \quad \forall k \in K \quad (3)$$

$$\sum_{i \in V'} x_{i0k} = 1 \quad \forall k \in K \quad (4)$$

$$x_{iik} = 0 \quad \forall i \in V', \forall k \in K \quad (5)$$

$$e_{0k} = Q \quad \forall k \in K \quad (6)$$

$$e_{jk} \leq e_{ik} - (d_{ij} + q_j) x_{ijk} + M(1 - x_{ijk}) \quad \forall (i, j) \in A, j \in P, \forall k \in K \quad (7a)$$

$$e_{jk} \leq Q z_{jk} + M(1 - x_{ijk}) \quad \forall (i, j) \in A, j \in D', \forall k \in K \quad (7b)$$

$$e_{jk} \geq e_{ik} - d_{ij} x_{ijk} - M(1 - x_{ijk}) \quad \forall (i, j) \in A, j \in D', \forall k \in K \quad (7b')$$

$$e_{ik} \geq d_{i0} x_{i0k} \quad \forall i \in V' \setminus \{0\}, \forall k \in K \quad (7c)$$

$$e_{ik} \leq Q \quad \forall i \in V', \forall k \in K \quad (8)$$

$$\sum_{i \in V'} \sum_{k \in K} x_{ij^c k} \leq M y_j \quad \forall j \in D, \forall j^c \in D'_j \quad (9)$$

$$z_{jk} \leq \sum_{j^c \in D'_j} \sum_{i \in V'} x_{ij^c k} \quad \forall j \in D, \forall k \in K \quad (10)$$

$$z_{jk} \leq y_j \quad \forall j \in D, \forall k \in K \quad (11)$$

$$u_{ik} - u_{jk} + |V'| x_{ijk} \leq |V'| - 1 \quad \forall (i, j) \in A, i \neq j, i, j \neq 0, \forall k \in K \quad (12)$$

$$1 \leq u_{ik} \leq |V'| - 1 \quad \forall i \in V' \setminus \{0\}, \forall k \in K \quad (13)$$

$$\sum_{i \in V'} \sum_{p \in P} x_{ipk} \geq \sum_{j \in V'} x_{0jk} \quad \forall k \in K \quad (14)$$

$$x_{ijk} \in \{0, 1\}, y_j \in \{0, 1\}, z_{jk} \in \{0, 1\} \quad (15a)$$

$$e_{ik} \geq 0, u_{ik} \geq 0 \quad (15b)$$

The objective function minimizes total operational cost consisting of UAV travel cost and fixed dock-opening cost.

Constraints (1) ensure each pipeline node is visited exactly once across all UAVs. Constraints (2) enforce flow conservation for each UAV at every node. Constraints (3) and (4)

ensure each UAV departs from and returns to the depot exactly once. Constraint (5) eliminates self-loops. Constraint (6) initializes each UAV with full battery at the depot.

Constraints (7a) track battery consumption when visiting a pipeline node: the battery upon arrival at j cannot exceed the battery at the previous node i minus the travel and inspection energy. Constraints (7b) enforce the *full-recharge* policy: if UAV k arrives at dock copy j^c and charges ($z_{jk} = 1$), its departure battery is set to Q ; the Big-M term relaxes the constraint when the arc is not used. Constraints (7b') are the complementary lower-bound constraints that prevent the battery variable from taking arbitrarily small values when the dock is actually visited: the departure battery must be at least the arriving battery minus travel cost, ensuring $e_{jk} = Q$ holds tightly when $x_{ijk} = 1$ and $z_{jk} = 1$. Constraint (7c) ensures sufficient battery to return to the depot. Constraint (8) caps the battery at capacity Q .

Constraint (9) ensures a physical dock j must be opened ($y_j = 1$) whenever any of its copies $j^c \in D'_j$ is visited by any UAV. Constraints (10) and (11) link the charging indicator z_{jk} to actual dock visits and to the opening decision, respectively.

Constraints (12) and (13) are Miller–Tucker–Zemlin (MTZ) sub-tour elimination constraints applied over the expanded node set V' . Because each physical dock j is represented by m_j distinct copy nodes in V' , multiple visits to the same physical location are modelled as visits to *different* nodes; the MTZ constraints therefore no longer conflict with such routes and correctly eliminate only genuine sub-tours.

Constraint (14) is a *minimum workload* constraint ensuring that every active UAV performs at least one pipeline inspection. Since constraints (3) and (4) require every UAV to depart from and return to the depot, without this constraint a UAV could follow a degenerate route $0 \rightarrow j^c \rightarrow 0$ (visiting only a dock copy for a recharge without inspecting any pipeline node), which incurs travel and dock-opening costs without contributing any inspection work. Constraint (14) prevents this by requiring that the number of pipeline arcs used by UAV k is at least as large as the number of arcs leaving the depot for UAV k ; since the latter equals 1 by constraint (3), this forces each UAV to visit at least one node in P .

Constraints (15) define variable domains.

Chapter 4

Metaheuristic Algorithm

The proposed approach is a **hybrid memetic algorithm** that integrates a **Genetic Algorithm (GA)** for global search, a **greedy dock insertion heuristic** for battery feasibility enforcement, and **local search (LS)** operators for route refinement. By combining population-based global exploration with problem-specific local improvement, the framework constitutes a memetic algorithm [1].

Infeasible solutions are permitted during the search and penalized in the fitness function based on cumulative battery deficit and idle UAV usage. A greedy dock insertion heuristic dynamically repairs routes by inserting recharging stops whenever the battery would otherwise be insufficient. Solution quality is further improved by local search operators applied periodically to elite individuals. All UAV routes start and end at the depot.

The penalized fitness function used throughout the algorithm is:

$$f(s) = \sum_{k \in K} \sum_{(i,j) \in A} c d_{ij} x_{ijk} + \sum_{j \in D} f_j y_j + \alpha \cdot \Delta_{\text{bat}}(s) + \beta \cdot \text{IdleUAV}(s) \quad (4.1)$$

where $\Delta_{\text{bat}}(s)$ is the total battery deficit accumulated across all UAV routes (the sum of negative battery values at each step, clamped to zero for feasible routes), $\text{IdleUAV}(s)$ is the number of UAVs assigned no pipeline nodes, $\alpha = 500$ is the battery penalty coefficient, and $\beta = 300$ is the idle UAV penalty coefficient. When $\Delta_{\text{bat}}(s) = 0$ and $\text{IdleUAV}(s) = 0$, the fitness reduces to the true objective value of the solution.

4.1 Solution Representation and Battery Model

4.1.1 Chromosome Representation

Each solution is represented as a set of $|K|$ ordered lists, one per UAV. Each list contains the sequence of nodes (pipeline and dock nodes) to be visited. Docking stations are *not* encoded directly in the chromosome; they are determined implicitly by the greedy dock insertion procedure applied after every genetic operation. The GA therefore searches over the space of *pipeline visitation sequences*, while battery feasibility is handled as a repair mechanism.

4.1.2 Battery Simulation and Full-Recharge Model

Battery levels are tracked by simulating each UAV route from the depot. The UAV departs with a full battery Q . At each pipeline node j , the battery is reduced by the travel distance d_{ij} and the inspection energy q_j :

$$e_j \leftarrow e_i - d_{ij} - q_j \quad (4.2)$$

At each docking station node j , the UAV recharges to *full capacity*:

$$e_j \leftarrow Q \quad (4.3)$$

This **full-recharge model** replaces the earlier partial-recharge formulation ($e_j \leftarrow \min(e_i - d_{ij} + R, Q)$). Under the full-recharge model, visiting the same physical dock twice in succession yields no benefit, as the first visit already restores the battery to Q . Any accumulated deficit (battery dropping below zero) is recorded as Δ_{bat} and penalized in the fitness function.

4.2 Initial Population

4.2.1 Greedy Construction

Each initial solution is constructed as follows:

1. Pipeline nodes are randomly shuffled.
2. Nodes are distributed among UAVs in a round-robin fashion, ensuring each UAV receives at least one pipeline node.
3. Within each UAV's assigned nodes, a **nearest-neighbor heuristic** orders them: starting from the depot, the closest unvisited pipeline node is selected at each step.

4. The **greedy dock insertion** procedure (Section 4.3) is applied to make the route battery-feasible.

4.2.2 Diverse Initial Population

To prevent premature convergence, the initial population of size $N_{\text{pop}} = 80$ is seeded with *all non-empty subsets of dock combinations*. For each subset $S \subseteq D$, two greedy solutions are generated using only the docks in S . The remaining slots are filled with randomly constructed greedy solutions. The top 15 solutions by fitness are then refined with the exhaustive dock local search (Section 4.5.3) before the GA begins.

4.3 Greedy Dock Insertion

The greedy dock insertion procedure takes an ordered pipeline sequence and inserts docking stops to ensure battery feasibility. For each pipeline node in sequence, the algorithm checks whether the UAV can (i) reach the node and (ii) return to the depot afterward. If either condition fails, a dock is inserted according to the following two-pass strategy:

Dock-before insertion. For each candidate dock $d \in D$, the algorithm evaluates inserting d *before* the current pipeline node j (i.e., route $\dots \rightarrow \text{prev} \rightarrow d \rightarrow j \rightarrow \dots$). The dock is selected if:

- The UAV can reach d from prev without the battery going negative,
- After full recharge at d , it can reach j and complete the inspection ($e_j = Q - d_{dj} - q_j \geq 0$),
- The remaining battery after visiting j suffices to return to the depot ($e_j \geq d_{j,0}$).

The dock minimizing the detour-plus-fixed-cost score $\delta(d) = (d_{\text{prev},d} + d_{d,j} - d_{\text{prev},j}) + f_d$ is chosen.

Dock-after insertion. If no feasible dock-before candidate exists, the algorithm tries inserting a dock *after* the current pipeline node j (i.e., $\dots \rightarrow j \rightarrow d \rightarrow \dots$). The dock is selected if the UAV can reach d from j , and after recharging to Q , can reach the depot ($Q \geq d_{d,0}$). The dock minimizing $d_{j,d} + f_d$ is selected.

Multiple recharges. The procedure naturally supports multiple dock visits within a single route: after each dock insertion the battery resets to Q , and the check continues for the remaining pipeline nodes. This allows routes such as $0 \rightarrow D_1 \rightarrow P_2 \rightarrow P_3 \rightarrow D_1 \rightarrow P_4 \rightarrow 0$, where the same physical dock is visited twice at genuinely different points in the route.

4.4 Genetic Algorithm Operators

4.4.1 Selection

Tournament selection with tournament size $k_{\text{tourn}} = 5$ is used. The top $E = 8$ solutions are preserved as elites in each generation.

4.4.2 Crossover: Order Crossover (OX)

Order Crossover (OX) operates on the *flat sequence of pipeline nodes* extracted from both parents (docking nodes are excluded). Two cut points a and b are chosen uniformly at random. The segment $[a, b]$ from parent 1 is copied directly to the child. The remaining positions are filled in the order they appear in parent 2, skipping nodes already present. This preserves relative ordering of pipeline visits while allowing recombination of route structure. Two children are produced symmetrically. After crossover, greedy dock insertion is re-applied to produce feasible routes.

4.4.3 Mutation

Five mutation operators are applied with probability $p_{\text{mut}} = 0.40$, each selected uniformly at random:

1. **Swap:** Two randomly selected pipeline nodes are swapped, possibly across different UAV routes.
2. **Relocate:** A single pipeline node is removed from its current route and reinserted at a random position in a (possibly different) UAV route.
3. **2-opt segment reversal:** A random contiguous segment of the current route is reversed in place.
4. **Or-opt (block move):** Two consecutive pipeline nodes are removed from their route and reinserted as a block into a random position in a (possibly different) route.
5. **Dock removal:** A randomly selected docking node is removed from a route. Greedy dock insertion is then re-applied to restore feasibility.

After every mutation, dock nodes are stripped from all routes and greedy dock insertion is re-applied to the pipeline-only sequences. This ensures that dock placements always reflect the current pipeline ordering rather than carrying over stale insertions from prior generations.

4.5 Local Search

Local search is applied every $L = 25$ generations to the top $E = 8$ elite solutions. Three operators are applied in sequence to each elite individual:

4.5.1 2-opt

For each UAV route, all pairs of pipeline nodes (i, j) with $j \geq i + 2$ are considered for segment reversal. The reversed sequence is passed through greedy dock insertion, and the move is accepted if the penalized fitness improves. The procedure repeats until no improvement is found for `max_no_imp` = 20 consecutive passes.

4.5.2 Relocate

All pipeline nodes across all UAV routes are considered for relocation. Each node is tentatively removed from its source route and inserted at every possible position in every UAV route (including the source route). The move yielding the greatest fitness improvement is accepted greedily. Routes are rebuilt via greedy dock insertion after each accepted move.

4.5.3 Exhaustive Dock Search

For each UAV route, the pipeline sequence is held fixed and all $2^{|D|}$ subsets of candidate docks are enumerated. For each subset, greedy dock insertion is applied using only the docks in that subset. The combination yielding the lowest penalized fitness is retained. This operator explicitly optimizes dock selection for a given pipeline ordering, decoupling the dock placement decision from the GA’s sequence search.

4.6 Diversification

If the best solution does not improve for `stagnation` = 80 consecutive generations, a **diversification restart** is triggered: the elite solutions are retained, and the rest of the population is replaced by a fresh diverse population generated using the same seeded construction described in Section 3.2. This prevents the search from becoming trapped in a local optimum while preserving the best solutions found so far.

4.7 Overall Algorithm

Algorithm 1 Hybrid Memetic GA for UAV Pipeline Inspection

```
1: Build diverse initial population of size  $N_{\text{pop}}$ :
2:   Generate greedy solutions seeded with all dock subsets  $S \subseteq D$ 
3:   Fill remaining slots with random greedy solutions
4:   Apply exhaustive dock search to top 15 solutions
5: Evaluate penalized fitness  $f(s)$  for all individuals
6: for generation  $g = 1$  to  $G_{\text{max}}$  do
7:   Preserve top  $E$  elite solutions
8:   while new population not full do
9:     Select two parents via tournament selection (size  $k_{\text{tourn}}$ )
10:    Apply Order Crossover (OX) on pipeline sequences
11:    Apply mutation with probability  $p_{\text{mut}}$  (one of: swap, relocate, 2-opt, or-opt, dock
        removal)
12:    Re-apply greedy dock insertion to offspring routes
13:    Evaluate  $f(s)$  and add offspring to new population
14:  end while
15: Sort population by  $f(s)$ ; update best solution found
16: if  $g \bmod L = 0$  then
17:   for each elite solution  $s$  do
18:     Apply 2-opt local search
19:     Apply relocate local search
20:     Apply exhaustive dock search
21:   end for
22:   Re-sort population
23: end if
24: if no improvement for stagnation generations then
25:   Retain elite solutions; replace rest with fresh diverse population
26: end if
27: end for
28: return best feasible solution found
```

The parameter values were determined through empirical testing on the sample instance ($|P| = 4$, $|D| = 3$, $|K| = 2$). A population of 80 with 400 generations was found to be sufficient, as solutions consistently converged within the first 100–150 generations across all random seeds tested. The mutation rate of 0.40 is intentionally higher than typical GA settings: because dock nodes are stripped and re-inserted after every genetic operation, the effective search perturbation is larger than the pipeline-sequence mutation alone, and

Table 4.1: GA parameter settings

Parameter	Symbol	Value	Rationale
Population size	N_{pop}	80	Sufficient diversity without excessive run-time
Number of generations	G_{max}	400	Convergence observed before generation 400 in all test runs
Elite size	E	8	Top 10% preserved; balances exploitation and diversity
Tournament size	k_{tourn}	5	Moderate selection pressure; avoids drift and over-elitism
Mutation rate	p_{mut}	0.40	Higher rate needed due to greedy dock re-insertion after every mutation
Local search interval	L	25	Frequent enough to refine elites without dominating runtime
Battery penalty	α	500	Exceeds maximum feasible objective; strongly penalises infeasibility
Idle UAV penalty	β	300	Penalises unbalanced load; kept below battery penalty
Stagnation threshold	stagnation	80	Restarts diversification after $\sim 20\%$ of budget with no improvement

a higher rate counteracts the tendency to over-exploit the greedy dock heuristic. The penalty coefficients $\alpha = 500$ and $\beta = 300$ were set to exceed the maximum observed feasible objective value, ensuring that any infeasible or unbalanced solution is ranked below all feasible ones. The stagnation threshold of 80 generations triggers a diversification restart when the search stalls, injecting fresh solutions seeded with different dock subsets while retaining the current elite, which consistently recovered solution quality in multi-seed experiments.

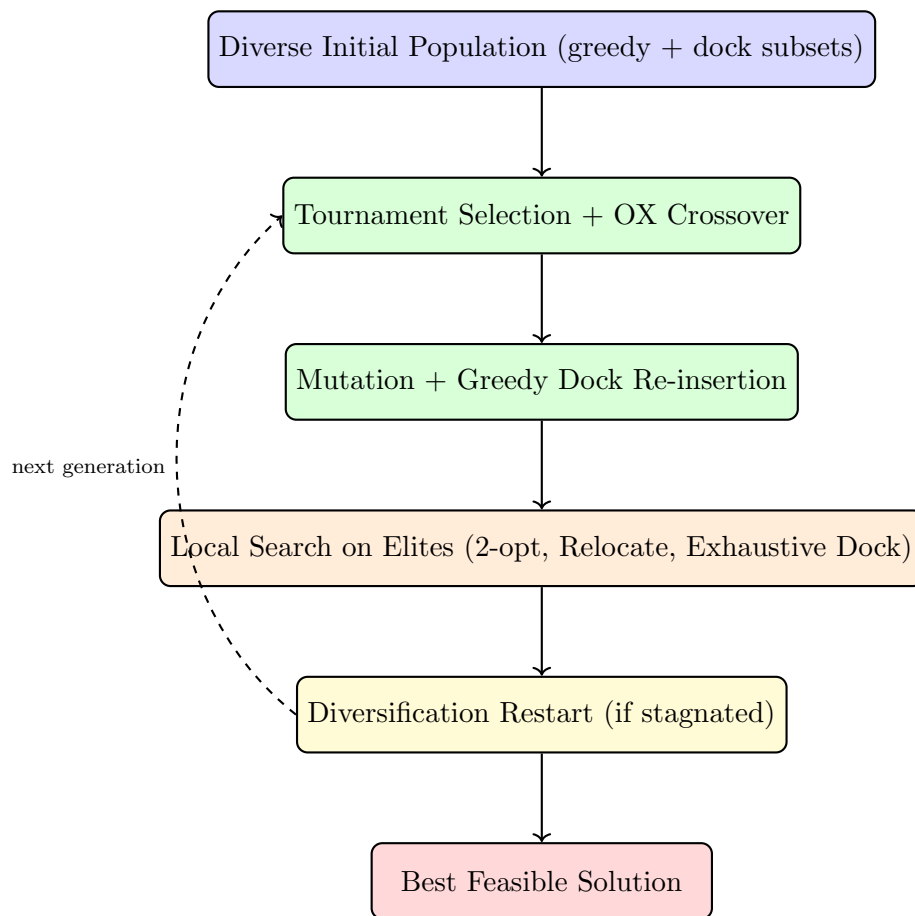


Figure 4.1: Structure of the hybrid memetic algorithm. Dashed arrow indicates the main generational loop.

Chapter 5

Experimental Analysis

5.1 Experimental Setup

All experiments were conducted on a single workstation running IBM ILOG CPLEX 12.10.0 via OPL. Instances were solved sequentially, one at a time. Small and medium instances (S1–M3) were solved without a time limit and terminated when the MIP optimality gap reached zero (or fell below 0.01%). Large instances (L1–L3) were subject to a **15-minute wall-clock time limit** (`cplex.tilim=900`); the best incumbent found at termination is reported.

The Genetic Algorithm (GA) was run with the following parameters: population size 80, 400 generations, mutation rate 0.40, elite size 8, tournament size 5, local search applied every 25 generations, and a stagnation restart triggered after 80 non-improving generations. Seven independent seeds {42, 7, 13, 99, 2024, 31, 55} were used per instance; the best objective and descriptive statistics across seeds are reported. Travel cost per unit distance is $c = 1.0$ with inspection energy $q_i = 3.0$ per pipeline node.

5.2 Benchmark Instances

Nine benchmark instances are organized into three size groups: *Small* (S1–S3: $|P| \leq 4$, $|D| \leq 3$, $K = 2$), *Medium* (M1–M3: $|P| \leq 10$, $|D| \leq 4$, $K \leq 3$), and *Large* (L1–L3: $|P| \leq 20$, $|D| \leq 6$, $K \leq 5$). The depot is fixed at the origin; battery capacity Q increases with scale. The MILP uses node-splitting with `maxVisits` = 2 copies per dock, yielding an expanded node set of size $|V'| = 1 + |P| + 2|D|$. Table 5.1 summarises all instance characteristics.

Table 5.1: Benchmark instance characteristics. $|P|$: pipelines; $|D|$: candidate docks; K : number of UAVs; Q : battery capacity; $|V'|$: expanded node set.

Group	Instance	$ P $	$ D $	K	Q	$ V' $
Small	S1	3	2	2	40	8
	S2	4	2	2	40	9
	S3	4	3	2	40	11
Medium	M1	6	3	2	60	13
	M2	8	4	3	60	17
	M3	10	4	3	70	19
Large	L1	12	5	4	80	23
	L2	15	5	4	80	26
	L3	20	6	5	100	33

5.3 Computational Results

Table 5.2 presents the main results. For each instance we report the CPLEX optimal value (or best incumbent under the time limit, marked $\dagger$), the best and mean GA objective across seven seeds, the MM-GA optimality gap Δ , and the wall-clock times for both methods. Δ is defined as

$$\Delta(\%) = \frac{z_{\text{GA}}^{\text{best}} - z^*}{z^*} \times 100,$$

where z^* is the CPLEX value (or best incumbent). A positive Δ indicates the GA solution is more expensive.

Table 5.2: Computational results: CPLEX vs. Genetic Algorithm. z^* : CPLEX optimal (or best incumbent †); $z_{\text{GA}}^{\text{best}}$ / $\bar{z}_{\text{GA}}$: best / mean GA objective over 7 seeds; Δ : MM-GA gap (%); t_C : CPLEX time (s); t_{GA} : total GA time for 7 seeds (s).

Group	Inst.	z^*	$z_{\text{GA}}^{\text{best}}$	$\bar{z}_{\text{GA}}$	$\Delta(\%)$	t_C (s)	t_{GA} (s)
Small	S1	110.19	114.76	114.76	+4.15	0.2	23.2
	S2	121.89	150.23	150.23	+23.25	0.2	30.7
	S3	116.03	140.48	141.64	+21.07	0.2	42.4
Medium	M1	154.02	174.14	174.14	+13.06	0.4	64.4
	M2	249.21	334.08	340.00	+34.06	17.6	106.6
	M3	298.95	382.35	391.26	+27.90	176.1	126.6
Large	L1 †	379.65	456.58	479.49	+20.26	900	175.0
	L2 †	358.12	480.28	488.07	+34.11	900	264.8
	L3 †	702.75	942.56	962.58	+34.13	900	483.7

† 15-minute time limit reached; residual MIP gaps: L1 22.86%, L2 16.78%, L3 25.72%.

5.4 Mathematical Model Performance

Small instances. CPLEX solves all three small instances to proven global optimality (gap = 0%) in under 0.25 seconds each. The MTZ-based formulation with the minimum-workload constraint (requiring every active UAV to visit at least one pipeline node) eliminates degenerate empty-route solutions and keeps the branch-and-cut tree manageable at this scale.

Medium instances. M1 is resolved optimally in 0.4s. M2 requires 17.6s and closes to a residual gap of 0.004%, effectively zero for practical purposes. M3 is the most demanding medium instance, taking 176.1s to reach a gap of 0.01%, again confirming near-certain global optimality. The sharp growth in solver time from M1 to M3 (0.4s $\rightarrow$ 176.1s) reflects the combinatorial explosion driven by additional pipeline nodes, dock copies, and UAV routes.

Large instances. For all three large instances the time limit is exhausted before the MIP gap closes. L1 terminates with a best incumbent of 379.65 (gap 22.86%), L2 with 358.12 (gap 16.78%), and L3 with 702.75 (gap 25.72%). Interestingly, L2 yields a lower incumbent than L1 despite having more pipeline nodes ($|P| = 15$ vs. $|P| = 12$), suggesting that the spatial layout of L2 admits more cost-efficient routing. The large residual gaps reflect the well-known difficulty of tightening the LP relaxation bound in MTZ-based vehicle-routing formulations as problem size grows.

5.5 Docking Station Utilisation

Table 5.3 reports the docks selected by CPLEX in its optimal solutions, illustrating how the model balances fixed infrastructure cost against routing detours.

Table 5.3: Docking station utilisation in CPLEX optimal solutions. d^* : number of docks opened; FC: total fixed-cost component.

Group	Inst.	d^*	FC	Opened docks
Small	S1	1	29.90	Dock 2
	S2	1	15.90	Dock 2
	S3	1	15.00	Dock 2
Medium	M1	1	30.40	Dock 1
	M2	1	26.30	Dock 2
	M3	2	46.40	Dock 1, Dock 3
Large	L1–L3	CPLEX did not reach optimality within the time limit		

A consistent pattern emerges: CPLEX selects the minimum number of docks required

for battery feasibility. Instances with tighter battery budgets or more dispersed pipelines require two open docks (M3); all others are served by a single dock.

5.6 Genetic Algorithm Performance

Solution quality and seed sensitivity. The GA produces feasible solutions on all nine instances across all seven seeds (100% feasibility rate). Table 5.4 summarises seed statistics.

Table 5.4: GA seed sensitivity (7/7 feasible seeds for every instance). $CV = \sigma/\bar{z} \times 100\%$.

Group	Inst.	z^{best}	$\bar{z}$	σ	CV (%)	Stability
Small	S1	114.76	114.76	0.00	0.00	Excellent
	S2	150.23	150.23	0.00	0.00	Excellent
	S3	140.48	141.64	1.83	1.29	Excellent
Medium	M1	174.14	174.14	0.00	0.00	Excellent
	M2	334.08	340.00	4.89	1.44	Good
	M3	382.35	391.26	3.82	0.98	Good
Large	L1	456.58	479.49	17.48	3.64	Acceptable
	L2	480.28	488.07	11.27	2.31	Good
	L3	942.56	962.58	17.37	1.80	Good

Small instances and M1 converge to a single solution regardless of seed ($CV = 0\%$), reflecting a near-unimodal fitness landscape. Variance first appears at M2 ($|P| = 8$, $K = 3$) and peaks at L1 ($CV = 3.64\%$), where the best seed (seed 7, $z = 456.58$) outperforms the worst (seed 42, $z = 513.78$) by 12.5%. All CV values remain below 4%, confirming robust GA behaviour across the benchmark. These results underscore the importance of multi-seed reporting: single-seed results can overstate typical GA performance by up to 12.5%.

Runtime. GA runtimes grow predictably from 23 s (S1) to 484 s (L3) and the algorithm always terminates without hitting any external budget. For medium instances the MM is faster by one to two orders of magnitude (e.g., 0.4 s vs. 64 s for M1); for large instances the runtime advantage reverses entirely in favour of the GA.

5.6.1 Scalability Analysis

Table 5.5 compares CPLEX and GA solve times across all nine instances as problem size increases.

As problem size grows, CPLEX solve times increase sharply: from 0.2 s for small instances to 176.1 s for M3, and eventually exceeding the 15-minute time limit for all large

Table 5.5: Scalability comparison. CPLEX time is a single run; GA time covers 7 seeds. †: 15-minute time limit reached.

Group	Inst.	t_{CPLEX} (s)	t_{GA} (s)	CPLEX status
Small	S1	0.2	23.2	Optimal
	S2	0.2	30.7	Optimal
	S3	0.2	42.4	Optimal
Medium	M1	0.4	64.4	Optimal
	M2	17.6	106.6	Optimal
	M3	176.1	126.6	Optimal
Large	L1	900 [†]	175.0	Time limit reached
	L2	900 [†]	264.8	Time limit reached
	L3	900 [†]	483.7	Time limit reached

instances. GA runtimes, by contrast, grow more gradually and predictably, ranging from 23.2 s (S1) to 483.7 s (L3). For small and medium instances CPLEX is significantly faster than the GA; however, for large instances the GA is the only method that consistently delivers a feasible solution within a practical time budget.

5.7 Comparative Discussion

Small instances (S1–S3). CPLEX proves optimality in under 0.2 s. The GA achieves 7/7 feasible seeds with $\text{CV} \leq 1.3\%$ but incurs gaps of 4–23%. The largest small-instance gap occurs at S2 (23.25%), where the GA selects a suboptimal dock and fails to exploit the cheaper dock-only routing structure found by CPLEX.

Medium instances (M1–M3). CPLEX remains optimal but solve time increases sharply: M3 takes 176 s and involves over 1 000 binary variables after presolve. GA gaps widen to 13–34%, indicating that the 2-opt and relocate operators alone are insufficient for the larger fleet-and-dock coordination problem. Seed variance is low ($\text{CV} \leq 1.44\%$), confirming that the GA landscape is consistent even if solution quality is suboptimal.

Large instances (L1–L3). CPLEX exhausts the 15-minute budget while still leaving gaps of 17–26%, confirming the NP-hard nature of the combined facility-location and battery-constrained routing problem at this scale. The GA remains the only practical solver, producing feasible solutions in 3–8 minutes (7 seeds total) with $\text{CV} \leq 3.64\%$. The best GA values (456.58, 480.28, 942.56 for L1–L3) constitute valid upper bounds; the true optima remain unknown.

Summary by group. Table 5.6 condenses the key trade-offs.

Table 5.6: Summary statistics by problem size group. [‡] M3 closed to gap 0.01%; treated as practically optimal.

Group	Avg z^*	Avg z_{GA}^{best}	Avg Δ (%)	Avg MIP gap	MM optimal
Small (S1–S3)	116.04	135.16	16.16	0.00%	3/3
Medium (M1–M3)	234.06	296.86	24.99	$\approx 0\%$	3/3 [‡]
Large (L1–L3)	480.17	626.47	29.50	21.8%	0/3

The MM is the method of choice for instances up to medium scale ($|P| \leq 10$), delivering proven optimal solutions within at most three minutes. For large-scale instances, the GA provides the only reliable fallback that guarantees feasibility and terminates without a hard cutoff. Hybridising the two approaches — using GA solutions as warm starts for CPLEX, or applying dedicated dock-assignment crossover operators to narrow the GA quality gap on medium instances — is a natural direction for future work.

5.8 Solution Visualisation: Instance S3

Instance S3 ($|P| = 4$, $|D| = 3$, $K = 2$, $Q = 40$) is selected for the comparative route visualisation because it is the smallest instance for which the full route detail of both the CPLEX optimum and the GA best solution is available, enabling a direct structural comparison. Figure 5.1 shows the two solutions side by side.

MM optimal solution ($z^* = 116.03$).

- **UAV 0:** Depot $\rightarrow$ P1 [bat = 25.82] $\rightarrow$ Depot.
- **UAV 1:** Depot $\rightarrow$ P4 [bat = 0] $\rightarrow$ D2 (*recharge*) $\rightarrow$ P3 [bat = 0] $\rightarrow$ D2 (*recharge*) $\rightarrow$ P2 [bat = 20.1] $\rightarrow$ Depot.

One dock (D2, fixed cost 15.00) is opened. Total travel cost: 101.03; total objective: **116.03**.

GA best solution ($z_{GA}^{\text{best}} = 140.48$, seed 7).

- **UAV 0:** Depot $\rightarrow$ P1 [bat = 25.8] $\rightarrow$ Depot.
- **UAV 1:** Depot $\rightarrow$ D5 (*recharge*) $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ D6 (*recharge*) $\rightarrow$ P4 $\rightarrow$ D5 $\rightarrow$ Depot.

Two docks are opened: D5 (cost 17.0) and D6 (cost 18.0), totalling 35.00 — more than double the MM’s dock expenditure. Total travel cost: 105.48; total objective: **140.48**.

Analysis. The travel cost difference is minor (+4.3%), but the dock cost difference (+133%) drives the entire 21.07% objective gap. This illustrates the core weakness of the GA in this benchmark: its evolutionary operators do not effectively co-optimize dock selection and routing simultaneously, leading to solutions that open unnecessary docking infrastructure.

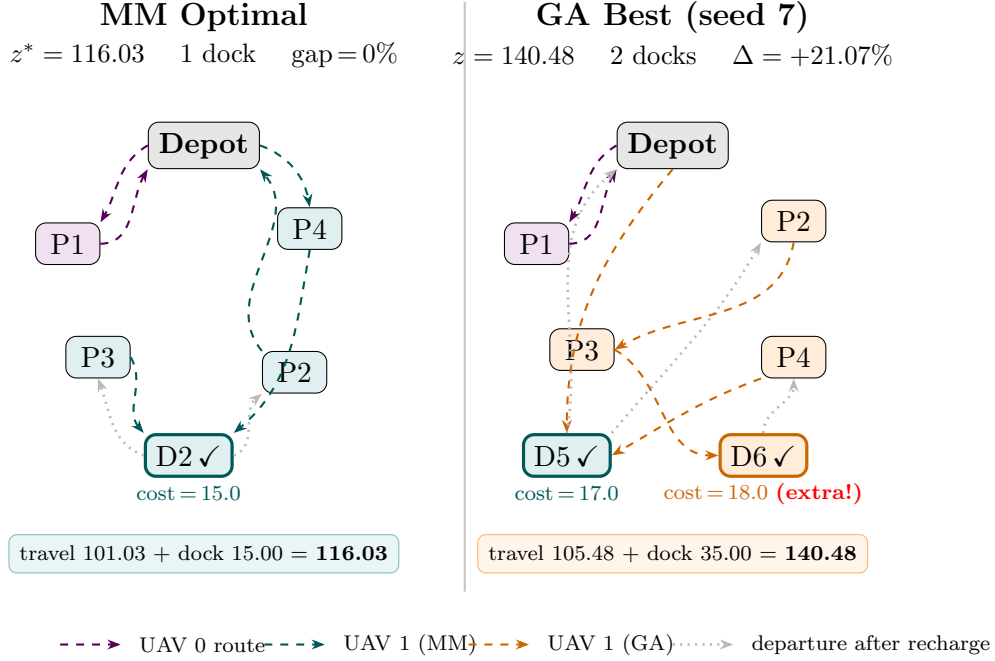


Figure 5.1: Side-by-side route maps for instance S3 ($|P| = 4$, $|D| = 3$, $K = 2$, $Q = 40$). *Left:* MM optimal ($z^* = 116.03$) — one dock (D2, cost 15) opened; UAV 1 recharges twice at the same dock. *Right:* GA best ($z = 140.48$) — two docks (D5+D6, total cost 35) opened unnecessarily. Travel costs differ by only 4.3% (101.03 vs. 105.48), but the dock cost difference (+133%) is solely responsible for the 21.07% gap in total objective. Dotted arcs denote UAV departure after a recharge event.

Chapter 6

Conclusion

This study addressed the UAV-based pipeline inspection problem, in which a fleet of unmanned aerial vehicles must inspect a set of spatially distributed pipeline nodes while respecting battery capacity constraints and jointly deciding which docking stations to open for mid-route recharging. The problem was formulated as a Mixed-Integer Linear Program (MILP) using an MTZ-based subtour elimination scheme over an expanded node set that models multiple visits to the same physical dock through node-splitting. A minimum-workload constraint was incorporated to eliminate degenerate solutions in which a UAV departs the depot without visiting any pipeline node. In parallel, a Genetic Algorithm (GA) was developed as a scalable heuristic alternative, evaluated across seven independent seeds per instance to ensure statistically meaningful comparisons.

Computational experiments on nine benchmark instances of increasing size demonstrated a clear trade-off between solution quality and computational scalability. The MILP formulation solved all small and medium instances (up to ten pipeline nodes) to proven global optimality within at most three minutes, confirming the effectiveness of the proposed model for moderate-scale deployments. For large instances (12–20 pipeline nodes), the exact solver exhausted a 15 minute time budget while leaving residual MIP gaps of 17–26%, highlighting the inherent computational difficulty of the combined facility-location and battery-constrained routing problem at scale. The GA, by contrast, delivered feasible solutions for every instance across all seven seeds with a 100% feasibility rate, terminating naturally within eight minutes regardless of instance size. However, GA solution quality lagged behind the MILP by an average of 16% on small instances, growing to approximately 30% on large instances, with the performance gap driven primarily by suboptimal dock selection rather than inefficient routing.

Several directions for future research emerge from these findings. First, the GA could be strengthened through dedicated dock-assignment crossover operators that co-optimize infrastructure selection and UAV routing simultaneously, targeting the primary source of suboptimality identified in this study. Second, a hybrid approach that uses GA solutions as warm starts for CPLEX — or applies exact local search post-processing to GA solutions

— could combine the scalability of the heuristic with the optimality guarantees of the exact method. Third, extending the model to consider heterogeneous UAV fleets, partial recharging policies, or stochastic battery consumption would bring the formulation closer to real-world operational conditions. Finally, applying stronger valid inequalities or Benders decomposition to the MILP could help close the optimality gap on large instances within practical time limits.

In conclusion, the proposed MILP formulation provides a rigorous and efficient tool for optimal UAV pipeline inspection planning at small to medium scale, while the GA offers a reliable and fast alternative for large-scale scenarios where exact optimisation is computationally prohibitive. Together, the two methods form a complementary framework that can be adapted to a wide range of real-world UAV inspection settings.

The source code for both the MILP model and the Genetic Algorithm, along with all benchmark instance data files, is publicly available at https://github.com/nazli-elif/EMU_676 to support reproducibility and future extensions of this work.

Bibliography

- [1] G. Kasparavičiūtė and M. Ludvigsen, “Genetic algorithm-based AUV mission optimisation with energy and priority constraints,” *OCEANS 2023-Limerick*, IEEE, 2023.
- [2] G. Kasparavičiūtė, K. Fagerholt, and M. Ludvigsen, “Energy-efficient route planning for optimizing underwater pipeline inspections using Resident Autonomous Underwater Vehicles,” *Ocean Engineering*, vol. 315, 2025, p. 119756.
- [3] A. M. Yazdani et al., “Real-time quasi-optimal trajectory planning for autonomous underwater docking,” *2015 IEEE International Symposium on Robotics and Intelligent Sensors (IRIS)*, IEEE, 2015.
- [4] V. De Carolis, K. E. Brown, and D. M. Lane, “Runtime Energy Estimation and Route Optimization for Autonomous Underwater Vehicles,” *IEEE Journal of Oceanic Engineering*, vol. 43, no. 3, pp. 608-623, 2018.
- [5] J. Zhang, M. Liu, S. Zhang, and R. Zheng, “Robust global route planning for an autonomous underwater vehicle in a stochastic environment,” *Frontiers of Information Technology & Electronic Engineering*, vol. 23, no. 11, pp. 1658-1672, 2022.